

Algorithms Design and Analysis

D&C Sheet

TABLE OF CONTENTS

TABLE OF CONTENTS.....	
FIRST: SOLVED PROBLEMS	
QUESTION1: Unimodal Search	
QUESTION2: Powering a Number	
QUESTION3: Celebrity Person	
QUESTION4: Quicksort Variation.....	
QUESTION5: Mergesort Variation	
QUESTION6: Bounding Box of Convex Polygon	
QUESTION7: $A[i] = i?$	
QUESTION8: Guess a Number!.....	
SECOND: UNSOLVED PROBLEMS	
Problem1: Position of Largest Element	
Problem2: Equivalent Strings	
THIRD: Tracing Questions	
QUESTION1: Quicksort.....	

Algorithms Design and Analysis

D&C Sheet

FIRST: SOLVED PROBLEMS

QUESTION1: Unimodal Search

An array $A[1...N]$ is unimodal if it consists of an increasing sequence followed by a decreasing sequence, or more precisely, if there is an index $m \in \{1, 2, \dots, N\}$ such that

- $A[i] < A[i + 1] \forall 1 \leq i < m$, and
- $A[i] > A[i + 1] \forall m \leq i < N$

In particular, $A[m]$ is the maximum element, and it is the unique “locally maximum” element surrounded by smaller elements ($A[m - 1]$ and $A[m + 1]$)

Give an algorithm to compute the maximum element of a unimodal input array in an efficient way?

Algorithms Design and Analysis

D&C Sheet

SOLUTION

D&C Steps

1. **Divide:** Check middle element with its adjacent.
2. **Conquer:** Recursively search 1 sub-array.
3. **Combine:** Trivial.

Pseudocode

```
UNIMODAL-SEARCH(A, low, high)
    If low == high then
        return low

    mid = low + (high - low)/2
    if A[mid] > A[mid-1] and A[mid] > A[mid+1] then
        return mid
    if A[mid] > A[mid-1] then
        return UNIMODAL-SEARCH(A, mid + 1, high)
    else
        return UNIMODAL-SEARCH(A, low, mid - 1)
    end if
```

Analysis

1. Calculate running time (recurrence relation)

$$T(N) = T(N/2) + O(1)$$

2. Solve it using master method:

- $a = 1, b = 2, f(N) = 1$
- Compare $f(N)$ vs. $N^{\log_b a}$
- $N^{\log_b a} = N^{\log_2 1} = N^0 = 1$ vs. $f(N) = 1$
- $N^{\log_b a} = \theta(f(N)) \rightarrow$ It's **case (2)**,

$$T(N) = \theta(\log(N))$$

Algorithms Design and Analysis

D&C Sheet

QUESTION2: Powering a Number

Write an efficient algorithm to compute A^N , N is positive integer without using "Power" function?

SOLUTION

Powering a Number (Normal Solution)

POWER-A-NUMBER (A, N)

 Pow = 1

For I = 1 **to** N

 Pow = Pow × A

End for

Return Pow

$\Theta(N)$

Algorithms Design and Analysis

D&C Sheet

Powering a Number (D&C)

D&C Steps:

1. **Divide:** Trivial.
2. **Conquer:** Recursively solve 1 part only with $N/2$.
3. **Combine:** Mul. the solution of subproblem by itself (and by A if odd).

Pseudocode:

```
POWER-A-NUMBER (A, N)
    If N = 1 then
        Return A
    pow = POWER-A-NUMBER (A, floor (N/2))
    if N % 2 ≠ 0 then
        return pow × pow × A
    else
        return pow × pow
    end if
```

Analysis: similar to previous problem

1. Calculate running time (recurrence relation)

$$T(N) = T(N/2) + O(1)$$

2. Solve it using master method:

- $a = 1, b = 2, f(N) = 1$
- Compare $f(N)$ vs. $N^{\log_b a}$
- $N^{\log_b a} = N^{\log_2 1} = N^0 = 1$ vs. $f(N) = 1$
- $N^{\log_b a} = \theta(f(N)) \rightarrow$ It's **case (2)**,

$$T(N) = \theta(\log(N))$$

Algorithms Design and Analysis

D&C Sheet

QUESTION3: Celebrity Person

Adel is a person whom everybody knows but he knows nobody. You have gone to a party. There are total n persons in the party. Your job is to find Adel in the party. You can ask questions of the form "Does person X know person Y?" How many questions you should ask to find Adel. ($O(n)$, $O(n^2)$, $O(n \log n)$,).

SOLUTION

Pseudocode:

Persons: is an Array Containing Persons

In main function:

```
X= Persons[1]           ; Initialization X to be first person.
Persons = Persons - {X} ; Remove it from the Persons List.
Adel = GetAdel(Persons, X)
```

```
GetAdel(Persons, X) returns Adel    //function that returns Adel
{
    If (length(Persons) == 0)
    {
        //No more persons to check against them → X is Adel
        Return X
    }
    If (X knows Persons[1])
    {
        //Since Adel knows NOBODY → X is not Adel, continue with
        //Persons[1] after removing it from the list
        GetAdel(Persons - {Persons[1]}, Persons[1])
    }
    Else
    {
        //Persons[1] is not Adel since Adel is known by everybody
        //Remove it from the list and continue with X
        GetAdel(Persons - {Persons[1]}, X)
    }
}
```

This code keep track of person X such that

1. Must **knows nobody**
2. Is **known by everybody**

→ So, by the end, when the list is finished, X must be Adel!! So this algorithm will give a correct solution

Algorithms Design and Analysis

D&C Sheet

Analysis:

From this code we can deduce that $T(N) = T(N - 1) + c$

Using Iteration method $\rightarrow \Theta(N)$

QUESTION4: Quicksort Variation

Suppose you have invented a new version of Quicksort, and you find, much to your surprise, that your new version always selects the second largest element of the list as the pivot point. What is the worst and average case of your new algorithm?

SOLUTION

Since the behavior of the algorithm is independent of the input, average and worst case are the same.

Worst case is given by the following recurrence.

$T(n) = T(n - 2) + (n - 1)$, Solve by iteration method

$\Rightarrow \Theta(N^2)$

QUESTION5: Mergesort Variation

Suppose your smart friend gives you a technique for merging two sorted arrays into one sorted array in constant time. Analyze the complexity of merge sort using your friend's technique?

SOLUTION

Since the work done by non-recursive part becomes $\Theta(1)$, the recurrence becomes:

$T(n) = 2 T(n/2) + \Theta(1)$,

Solve by Master method (case 1)

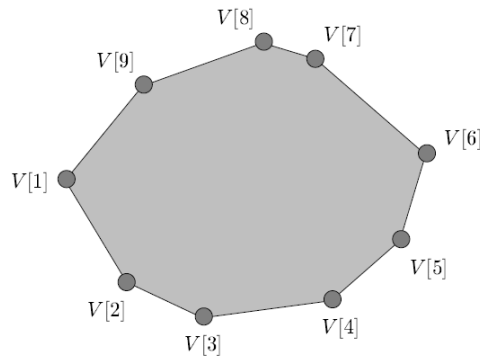
$\Rightarrow \Theta(N)$

QUESTION6: Bounding Box of Convex Polygon

A polygon is convex if all of its internal angles are less than 180° (and none of the edges cross each other). Figure 1 shows an example.

Algorithms Design and Analysis

D&C Sheet



We represent a convex polygon as an array $V[1...N]$ where each element of the array represents a vertex of the polygon in the form of a coordinate pair (x,y) .

We are told that $V[1]$ is the vertex with the minimum x-coordinate and that the vertices $V[1...N]$ are ordered counterclockwise, as in the figure. You may also assume that the x-coordinates of the vertices are all distinct, as are the y-coordinates of the vertices.

It's required to find the bounding box of such polygon in an efficient way? (i.e. minx, maxX, minY, maxY)

Summary:

- 1- All angles are less than 180
- 2- None of edges cross each other
- 3- $V[1...N]$ contains the vertices sorted in counter clockwise direction
- 4- $V[1]$ is the vertex with minimum X coordinate

Requirements:

1. Search for the vertex with maximum X coordinate in $O(\lg(N))$?
2. Search for the vertex with minimum Y coordinate in $O(\lg(N))$?
3. Search for the vertex with maximum Y coordinate in $O(\lg(N))$?

SOLUTION

Algorithms Design and Analysis

D&C Sheet

Convex Polygon

```
CONVEX-POLYGON(Vertices, N)
    index = UNIMODAL-SEARCH-X(Vertices[1...N])
    Max-X = Vertices[index].X
    index = UNIMODAL-SEARCH-Y(Vertices[index...N])
    if (Vertices[index].Y > Vertices[1].Y)
        Max-Y = Vertices[index].Y
    else
        Max-Y = Vertices[1].Y

... and so on
```

Algorithms Design and Analysis

D&C Sheet

QUESTION7: $A[i] = i$?

Suppose you are given an increasing sequence of n distinct integers in an array $A[1..n]$. Design an efficient algorithm to determine whether there exists an index i such that $A[i] = i$. Analyze the running time of your algorithm.

SOLUTION

Idea of the algorithm

The algorithm starts from the middle of the array A and check if we have found a match (i.e. $A[mid] = mid$). If not, we check to see if what we've got is greater or lesser than the required value. If it is greater (i.e. $A[mid] > mid$), then we can confidently ignore all the values greater than and including the *midpoint*. If it is smaller (i.e. $A[mid] < mid$), then we can ignore all the values smaller than and including the *midpoint*. This goes on until we either find a match or when the *low* point is greater than

Details

```
Check_Existence(array A, int low, int high)
{
    if (low > high)
        return false;
    mid = floor((low + high)/2);
    if (A[mid] == mid)
        return true;
    else if (A[mid] > mid)
        return Check_Existence(A, low, mid-1);
    else
        return Check_Existence(A, mid+1, high);
}
```

It is like binary search except the condition. In binary search we are searching for $A[mid] = \text{key}$. But here we are searching for $A[mid] = mid$.

From this code we can deduce that $T(N) = T(N/2) + c$;

Master Method:

$F(N)$ vs $N^{\log_b a}$

1 vs $N^{\log_2 1}$

1 vs 1 $\Rightarrow$ it is $\Theta(\log N)$ (case 2)

Algorithms Design and Analysis

D&C Sheet

QUESTION 8: Guess a Number!

Your friend guesses an integer between A and B: You can ask questions like is the number less than 100? He will give YES NO answers. How many questions can your friend force you to ask, if you are a smart person?

SOLUTION

From the problem statement, our guess will be in the form " is the number less than X".

(Hint: This is a game that most of us play it ..Beta3et "اخترك رقم")

Details

```
int FindNumber (int low, int high)
{
    if (low == high)
        return low;
    mid = floor((low + high)/2);
    if (Is the number less than mid? == true)
        return FindNumber(low, mid-1);
    else
        return FindNumber(mid, high);
}
```

As shown it also has same behavior as binary search except the conditions so.

Let $N = B - A$

$T(N) = T(N/2) + c$ which is $\Theta(\log N)$ BY Master Method

Algorithms Design and Analysis

D&C Sheet

SECOND: UNSOLVED PROBLEMS

Problem1: Position of Largest Element

Design a **divide-and-conquer** algorithm for finding the position of the largest element in an array of N numbers?

Problem2: Equivalent Strings

Two strings a and b of equal length are called equivalent in one of the two cases:

1. They are equal.
2. If we split string a into two halves of the same size a_1 and a_2 , and string b into two halves of the same size b_1 and b_2 , then one of the following is correct:
 1. a_1 is equivalent to b_1 , and a_2 is equivalent to b_2
 2. a_1 is equivalent to b_2 , and a_2 is equivalent to b_1

Sample Input1: aaba, abaa

Sample Output1: True

Sample Input2: aabb, abab

Sample Output2: false

Note

In the first sample you should split the first string into strings "aa" and "ba", the second one — into strings "ab" and "aa". "aa" is equivalent to "aa"; "ab" is equivalent to "ba" as "ab" = "a" + "b", "ba" = "b" + "a".

In the second sample the first string can be splitted into strings "aa" and "bb", that are equivalent only to themselves. That's why string "aabb" is equivalent only to itself and to string "bbaa".

THIRD: Tracing Questions

QUESTION1: Quicksort

For the Quick Sort algorithm (Ascending) with first element as a pivot, write down the **entire array** after **finishing** the **first level** (i.e. after placing 1st pivot in its correct location):
{89, 66, 58, 57, 52, 49, 100, 60}

Answer

60, 66, 58, 57, 52, 49, 89, 100